Predavanje 2_b

Poznato je da se u jeziku C niti jedan ozbiljniji problem ne može riješiti bez upotrebe nizova. Nizovi zaista jesu veoma korisne strukture podataka, koje se sasvim regularno mogu koristiti i u jeziku C++. Međutim, C je jezik *mnogo nižeg nivoa* od jezika C++ (u smislu da mu je filozofija razmišljanja više orijentirana ka načinu rada samog računara). Posljedica toga je da mnogi koncepti nizova iz jezika C nisu u skladu s naprednim konceptima jezika C++. Posmatrano s aspekta C++-a, nizovi preuzeti iz jezika C posjeduju brojne nedostatke i nedoslijednosti. Možemo reći da rad s C-ovskim nizovima, mada sasvim legalan, nije "u duhu" jezika C++. Da bi se ovi problemi izbjegli, u jezik C++ uveden je novi tip podataka, nazvan "`vector`", koji je definiran u istoimenom zaglavlju standardne biblioteke jezika C++ (tako da za korištenje ovog tipa podataka moramo uključiti u program zaglavlje biblioteke "`vector`", a također je potreban i prefiks "`std::`" ukoliko ne koristimo naredbu "`using`"). Ovaj tip podataka (zovimo ga prosto *vektor*) zadržava većinu svojstava koji posjeduju standardni nizovi, ali ispravlja neke njihove nedostatke. Doduše, naziv "vektor" za ovaj tip podataka nije baš najsretnije odabran, jer nema prevelike sličnosti s konceptom vektora u matematici i fizici (prikladniji bi naziv bio *fleksibilni niz*), ali tvorci jezika C++ odlučili su se za ovaj naziv, tako da to moramo prihvatiti, sviđalo nam se to ili ne. Promjenljive tipa "`vector`" mogu se deklarirati na desetak različitih načina (ovisno od toga kakav želimo da bude njihov početni sadržaj nakon kreiranja), od kojih se najčešće koriste sljedeća tri načina:

```
std::vector<tip_elemenata> ime_promjenljive
std::vector<tip_elemenata> ime_promjenljive(inicijalni_broj_elemenata)
std::vector<tip_elemenata> ime_promjenljive(inicijalni_broj_elemenata, inicijalna_vrijednost)
```

Na primjer, vektor "`ocjene`", čiji su elementi cjelobrojni, možemo deklarirati na jedan od sljedećih načina:

```
std::vector<int> ocjene;
std::vector<int> ocjene(10);
std::vector<int> ocjene(10, 5);
```

Prva deklaracija deklarira vektor "`ocjene`", koji je inicijalno prazan, odnosno ne sadrži niti jedan element (vidjećemo kasnije kako možemo naknadno dodavati elemente u njega). Druga deklaracija (koja se najčešće koristi) deklarira vektor "`ocjene`", koji inicijalno sadrži 10 elemenata, a koji su automatski inicijalizirani na vrijednost 0. Treća deklaracija deklarira vektor "`ocjene`", koji inicijalno sadrži 10 elemenata, a koji su automatski inicijalizirani na vrijednost 5 (tužan početak, zar ne?).

Primijetimo da "`vector`" *nije ključna riječ*, s obzirom da tip vektor *nije tip koji je ugrađen u samo jezgro jezika* C++, već izvedeni tip (slično kao i npr. tip "`complex`"), definiran u istoimenoj biblioteci "`vector`". Promjenljive tipa *vektor* mogu se u gotovo svim slučajevima koristiti poput običnih nizovnih promjenljivih (za pristup individualnim elementima vektora, kao i kod običnih nizova, koriste se uglaste zagrade). S druge strane, rad s ovakvim promjenljivim je znatno fleksibilniji nego s običnim nizovima, mada postoje i neki njihovi minorni nedostaci u odnosu na klasične nizove. U nastavku će biti opisane najbitnije razlike između običnih nizovnih promjenljivih i promjenljivih tipa *vektor*.

Na prvom mjestu, broj elemenata neke vektorske promjenljive nije *svojstvo samog tipa* koji određuje promjenljivu, nego njeno *individualno*, *dinamičko svojstvo same promjenljive*, koje se može mijenjati tokom njenog života. Naime, vektorska promjenljiva u toku svog života može povećavati i smanjivati broj svojih elemenata (samim tim, zauzeće memorije koje zauzima neka vektorska promjenljiva može se dinamički mijenjati, dok nizovi od trenutka svoje deklaracije do kraja postojanja zauzimaju uvijek istu količinu memorije). Broj elemenata pri deklaraciji vektora navodi se u *običnim*, a ne u uglastim zagradama, čime je istaknuta razlika između zadavanja broja elemenata i navođenja indeksa za pristup elementima. Na primjer, vektor "`ocjene`" od 10 elemenata deklarirali bismo kao

```
std::vector<int> ocjene(10);
```

dok bismo "obični" niz "`ocjene`" od 10 elemenata deklarirali kao:

```
int ocjene[10];
```

U slučaju običnih nizova, indeks unutar uglaste zagrade ima jedno značenje pri deklaraciji niza (ukupan broj elemenata niza), a sasvim drugo značenje pri upotrebi elemenata niza, kada predstavlja indeks elementa kojem se pristupa (npr. "`ocjene[3]`" je element niza "`ocjene`" s indeksom 3). Kod vektora ova dva značenja su jasno razdvojena: obične zagrade se koriste pri *zadavanju broja elemenata*,

a uglaste pri *izboru željenog elementa*. Bitno je napomenuti da ukoliko greškom zadamo deklaraciju s uglastim zagradama poput

```
std::vector<int> ocjene[10];        // Radi sintaksno, ali daje neočekivani efekat...
```

nećemo dobiti grešku, nego ćemo na taj način deklarirati *niz* (*obični*) *od 10 elemenata*, *čiji su elementi* (*inicijalno prazni*) *vektori*. Vidjećemo kasnije da su ovakve konstrukcije sasvim moguće, legalne i često korisne, ali u ovom trenutku to *nije ono što smo vjerovatno željeli*.

Nije na odmet napomenuti da se pri radu s vektorima također često koriste "**typedef**" deklaracije s ciljem pojednostavljenja sintakse i lakšeg kucanja. Tako se recimo nakon deklaracije

```
typedef std::vector<int> IntVektor;
```

riječ "IntVektor" može koristiti bilo gdje kao sinonim za konstrukciju "std::vector<int>". U nastavku ovog izlaganja ipak se nećemo koristiti ovom pogodnošću.

Poznato je da prilikom deklaracije nizova, broj elemenata niza mora biti (prava) konstanta. Drugim riječima, konstrukcije poput sljedeće nisu legalne u C++-u (niti u starijim dijalektima C-a):

```
int broj_ocjena;
std::cout << "Koliko ima ocjena: ";
std::cin >> broj_ocjena;
int ocjene[broj_ocjena];                // Ovo je po svim C++ standardima ilegalno!
```

Neki kompajleri, poput kompajlera iz GNU porodice kompajlera, doduše prihvataju ovakve deklaracije, ali treba znati da je to *nestandardno proširenje* (*ekstenzija*) *podržano od konkretnog kompajlera*, a ne dio standarda jezika C++ i vjerovatno *neće raditi pod drugim kompajlerima*. Mada su ovakve deklaracije postale legalne u novijim dijalektima jezika C (počev od C99 nadalje), komitet za standardizaciju jezika C++ je odlučio da *neće podržati* ovakve konstrukcije u jeziku C++, jer je trend da se u jeziku C++ korisnici podstiču da polako izbacuju nizove iz opće upotrebe i da ih zamjenjuju novim fleksibilnijim tipovima podataka poput vektora (dok u C-u nema jednostavne alternative korištenju nizova). Također u jeziku C su takvi nizovi čiji broj elemenata nije unaprijed poznat izvedeni na *prilično traljav način* (recimo, moguć je krah ukoliko se zatraži prevelik broj elemenata, pri čemu ne postoji način da saznamo kolika je maksimalno dozvoljena veličina), što je još jedan razlog zašto takvi nizovi nisu podržani u jeziku C++. S druge strane, kod vektora željeni broj elemenata može biti *proizvoljan izraz*, a ne samo konstantna vrijednost ili konstantan izraz, tako da ne mora biti apriori poznat (i to je podržano na kvalitetan način). Stoga je sljedeći programski isječak posve legalan:

```
int broj_ocjena;
std::cout << "Koliko ima ocjena: ";
std::cin >> broj_ocjena;
std::vector<int> ocjene(broj_ocjena);                    // Ovo je OK...
```

Sljedeća interesantna osobina vektora je što se svi njihovi elementi *automatski inicijaliziraju na podrazumijevanu vrijednost za tip elemenata vektora* (ta vrijednost je *nula* za sve brojčane tipove), a po želji možemo pri deklaraciji zadati i drugu vrijednost koja će biti iskorištena za inicijalizaciju. Ovo je bitno drugačije od običnih nizova, kod kojih elementi imaju nedefinirane vrijednosti, ukoliko se eksplicitno ne navede lista inicijalizatora (doduše, ovo vrijedi samo za nizove koji su definirani kao lokalne i nestatičke promjenljive, s obzirom da je još iz jezika C naslijeđena konvencija po kojoj se elementi globalno definiranih nizova ili nizova deklariranih s ključnom riječju "**static**" također automatski inicijaliziraju na nulu).

Razmotrimo sada kako možemo izvršiti inicijalizaciju elemenata vektora na proizvoljne početne vrijednosti. Znamo da su kod nizova moguće deklaracije poput sljedećih (u drugoj deklaraciji, nedostajuća dva elementa u listi se automatski dopunjavaju nulama, a u trećoj deklaraciji, dimenzija niza 5 se automatski izvodi iz dužine liste inicijalizatora):

```
int a[7] = {3, 5, 2, 6, 4, 3, 9};
int b[6] = {2, 4, 7, 3};                    // Nedostajući elementi su 0
int c[] = {2, 3, 5, 1, 2};                  // Automatsko određivanje veličine
```

Na sličan način se mogu inicijalizirati i vektori, recimo pomoću sljedećih konstrukcija:

```
std::vector<int> v = {3, 5, 2, 8, 6};       // Obje konstrukcije imaju
std::vector<int> v{3, 5, 2, 8, 6};          //   isti krajnji efekat...
```

Primijetimo da se u ovim konstrukcijama veličina vektora *ne navodi*. Inače, od prethodne dvije konstrukcije preporučuje se druga, iako su za mnoge tipove podataka, uključujući i vektore, one identične. Naime, za neke rijetke tipove podataka može biti razlike između ove dvije inicijalizacije, pri čemu se tada konstrukcija bez znaka "=" bolje ponaša, pa je bolje odmah se navići na nju. Također, već smo istakli da C++ podržava da se pri inicijalizaciji nizova može izostaviti znak "=", tako da je ranije navedene deklaracije (uz inicijalizaciju) nizova "a", "b" i "c" radi konzistencije preporučljivo pisati ovako:

```
int a[7]{3, 5, 2, 6, 4, 3, 9};                    // Preporučeno...
int b[6]{2, 4, 7, 3};                             // Također...
int c[]{2, 3, 5, 1, 2};                           // Također...
```

Jasno je da prilikom inicijalizacije tip elemenata u listi inicijalizatora mora biti ili isti kao tip elemenata vektora, ili mora biti moguća automatska pretvorba u tip elemenata vektora. Pri tome, degradirajuće pretvorbe *nisu dozvoljene* (npr. ukoliko je tip elemenata "**int**", a lista inicijalizatora sadrži realne brojeve), odnosno kompajler će u takvim slučajevima prijaviti grešku.

Bitno je uočiti razliku između sljedeće dvije, sintaksno slične, a s aspekta značenja vrlo različite deklaracije:

```
std::vector<int> v(10);                   // "v" ima 10 elemenata, koji su nule
std::vector<int> v{10};                   // "v" ima 1 element 10
```

Naime, prva konstrukcija kreira vektor "v" koji sadrži 10 elemenata, koji se inicijalno postavljaju na nule, dok druga konstrukcija kreira vektor "v" sa samo jednim elementom 10. Ovo je tipičan primjer deklaracija u kojima upotreba okruglih i vitičastih zagrada u deklaracijama nema isto značenje. Slično je s deklaracijama poput "`std::vector<int> v(10, 5)`" ("v" je vektor od 10 elemenata koji imaju inicijalnu vrijednost 5) i "`std::vector<int> v{10, 5}`" ("v" je vektor od 2 elementa 10 i 5).

Rekli smo već da kod vektora postoji mogućnost naknadnog mijenjanja broja njihovih elemenata, što se postiže primjenom funkcije "`resize`" nad vektorom, pri čemu se kao parametar zadaje novi broj elemenata (koji može biti veći ili manji od aktuelnog broja elemenata). Na primjer, ukoliko je vektor "`ocjene`" deklariran tako da prima 10 cijelih brojeva (ocjena), a naknadno se utvrdi da je potrebno zapisati 15 ocjena, moguće je izvršiti naredbu

```
ocjene.resize(15);
```

nakon koje će broj elemenata vektora "`ocjene`" biti povećan s 10 na 15. Postojeći elementi zadržavaju svoje vrijednosti, a novododani elementi inicijaliziraju se na podrazumijevane vrijednosti. Ukoliko želimo da sami zadamo na koje će vrijednosti inicijalizirati novododani elementi, to možemo uraditi zadavanjem dodatnog drugog parametra funkciji "`resize`". Na primjer, ukoliko želimo da novododani elementi dobiju vrijednost 7, to možemo uraditi pomoću naredbe

```
ocjene.resize(15, 7);
```

Sljedeća interesantna osobina vektora je mogućnost dodjeljivanja jednog vektora drugom pomoću operatora dodjele "=" (pri čemu se kopiraju *svi elementi*, odnosno vektor *kao cjelina*), što nije moguće s nizovima (ni u C-u, ni u C++-u). Na primjer, ukoliko su "`niz1`" i "`niz2`" dva niza istog tipa, nije moguće izvršiti dodjelu poput "`niz1 = niz2`" s ciljem kopiranja svih elemenata niza "`niz2`" u niz "`niz1`", nego je neophodno vršiti kopiranje element po element uz pomoć petlji, na primjer ovako (uz pretpostavku da je "`broj_elemenata`" zajednički broj elemenata oba niza):

```
for(int i = 0; i < broj_elemenata; i++) niz1[i] = niz2[i];
```

Alternativno je moguće koristiti neku bibliotečku funkciju (poput "`memcpy`") koja u načelu radi istu stvar. Međutim, ukoliko su "`v1`" i "`v2`" vektori čiji su elementi istog tipa, dodjela poput "`v1 = v2`" je sasvim legalna, pri čemu dolazi do kopiranja svih elemenata vektora "`v2`" u vektor "`v1`" (na brz i efikasan način). Vektori "`v1`" i "`v2`" pri tom čak ne moraju imati ni isti broj elemenata: veličina vektora "`v1`" se automatski modificira tako da nakon obavljenog kopiranja vektor "`v1`" ima isti broj elemenata kao i vektor "`v2`". Ipak, postoji ograničenje da tip elemenata vektora "`v1`" i "`v2`" *mora biti potpuno isti* da bi dodjela poput "`v1 = v2`" bila prihvaćena. U suprotnom, dobićemo grešku pri kompajliranju čak i ukoliko se tip elemenata vektora "`v2`" može automatski konvertirati u tip elemenata vektora "`v1`" (npr. ukoliko su elementi vektora "`v2`" tipa "**int**", a vektora "`v1`" tipa "**double**"). U tom slučaju, preostaje nam jedino da ručno kopiramo elemente jedan po jedan petljom, ili da koristimo bibliotečku funkciju "`copy`" o kojoj ćemo kasnije govoriti.

U jeziku C++, lista inicijalizatora može se iskoristiti ne samo za *inicijalizaciju*, nego i za *dodjelu* vektorima, odnosno ranije kreiranom vektoru je moguće *dodijeliti listu inicijalizatora*, pod uvjetom da se tip elemenata u listi slaže s tipom elemenata vektora, ili da je moguća njegova automatska pretvorba u tip elemenata vektora. Pri tome se raniji sadržaj vektora briše, njegova veličina se prilagođava veličini liste inicijalizatora, a elementi liste se kopiraju u vektor. Recimo, ukoliko je "`v`" neki vektor čiji su elementi tipa "**`int`**", sljedeća dodjela je sasvim smislena, nakon čega će vektor "`v`" imati 5 elemenata 3, 5, 1, 2 i 6, neovisno od toga kakav je bio njegov raniji sadržaj:

```
v = {3, 5, 1, 2, 6};
```

U jeziku C postoji tijesna veza između nizova i pokazivača iskazana kroz dvije činjenice. Prva je da se ime niza upotrijebljeno samo za sebe bez uglastih zagrada *automatski konvertira u pokazivač na prvi element niza* (uz jedini izuzetak kada je ime niza upotrijebljeno kao argument operatora "**`sizeof`**" ili "`&`", kada ta pretvorba ne vrijedi). Stoga, ukoliko je "`ocjene`" neki niz, tada se ime "`ocjene`" upotrijebljeno samo za sebe (tj. bez indeksiranja) interpretira kao "`&ocjene[0]`". Ova osobina se u jeziku C, zajedno s pokazivačkom aritmetikom, često koristi da se neke operacije izvedu na efektniji način. Svi ovi trikovi rade i u jeziku C++. Na primjer, ukoliko je "`ocjene`" niz od 10 cijelih brojeva, njegove elemente možemo ispisati i sljedećim programskim isječkom, u kojem se malo "igramo" s pokazivačkom aritmetikom (analizirajte pažljivo ovaj isječak, da se podsjetite pokazivačke aritmetike):

```
for(int *p = ocjene; p < ocjene + 10; p++) std::cout << *p << std::endl;
```

Druga činjenica koja iskazuje vezu između pokazivača i nizova iskazuje se u činjenici da se indeksiranje može primijeniti i na pokazivače, pri čemu se u slučaju da je "`p`" pokazivač, a "`i`" cijeli broj, izraz oblika "`p[i]`" interpretira kao "`*(p + i)`", ali ta činjenica nije bitna za ono o čemu ovdje želimo govoriti. Bitno je to da se imena vektora upotrijebljena sama za sebe, bez indeksa, za razliku od nizova *ne konvertiraju automatski* u pokazivač na prvi element vektora. Zbog toga, prethodni isječak *neće biti sintaksno ispravan* u slučaju da "`ocjene`" nije niz nego vektor. To ne znači da se pokazivačka aritmetika ne može koristiti s vektorima. Za elemente vektora se *garantira* da su u memoriji smješteni jedan iza drugog, u rastućem poretku indeksa, tako da je pokazivačka aritmetika valjana i s elementima vektora (iznimka su jedino vektori čiji su elementi tipa "**`bool`**", o čemu će biti riječi malo kasnije). Jedino je adresu elemenata vektora neophodno uzeti *eksplicitno*, zbog nepostojanja automatske konverzije. Stoga bismo prethodni isječak mogli prepraviti ovako da radi s vektorima:

```
for(int *p = &ocjene[0]; p < &ocjene[0] + 10; p++) std::cout << *p << std::endl;
```

Isječci poput ovako prepravljenog isječka radiće korektno i s nizovima i s vektorima (osim ukoliko su elementi vektora tipa "**`bool`**", što smo već nagovijestili). Mada nekome nepostojanje automatske konverzije vektora u pokazivač na prvi element vektora može djelovati kao nedostatak a ne kao prednost, treba napomenuti da su na taj način eliminirane brojne misinterpretacije, kao što ćemo uskoro vidjeti. Neko bi mogao postaviti pitanje da li se umjesto izraza "`&ocjene[0] + 10`" moglo pisati prosto "`&ocjene[10]`". Mada će to na većini kompajlera dati ispravno ponašanje, tehnički gledano izraz "`&ocjene[10]`" je *nelegalan* (predstavlja *nedefinirano ponašanje*), jer uzimamo adresu *nepostojećeg elementa vektora* "`ocjene[10]`". S druge strane, izraz "`&ocjene[0] + 10`" je *legalan*, s obzirom da je pomoću pokazivačke aritmetike *legalno kreirati pokazivač koji pokazuje tačno iza posljednjeg elementa niza ili vektora* (ali ne i dalje od toga).

Razmotrimo sada problem poređenja dva niza odnosno vektora. Ukoliko želimo ispitati da li su dva niza jednaka odnosno različita (pri čemu smatramo da su nizovi jednaki ukoliko su im jednaki elementi s istim indeksima), to ne možemo uraditi uz pomoć operatora "`==`" odnosno "`!=`". Što je najgore, ukoliko su "`niz1`" i "`niz2`" dva niza istog tipa, izrazi poput "`niz1 == niz2`" i "`niz1 != niz2`" su savršeno legalni, ali *ne rade ono što se očekuje*. Naime, zbog automatske konverzije nizova u pokazivače, u ovakvim izrazima se zapravo uspoređuju *adrese prvih elemenata nizova*, a ne sami nizovi. Ovo je tipičan primjer situacije u kojoj automatska konverzija nizova u pokazivače može dovesti do zabune. Za poređenje nizova moramo ponovo koristiti konstrukcije koje koriste petlje, kao u sljedećem primjeru:

```
bool isti_su = true;
for(int i = 0; i < broj_elemenata; i++)
  if(niz1[i] != niz2[i]) {
    isti_su = false; break;
  }
if(isti_su) std::cout << "Nizovi su isti!";
else std::cout << "Nizovi su različiti!";
```

S druge strane, ukoliko su "`v1`" i "`v2`" vektori, njihovo poređenje može se slobodno izvršiti uz pomoć operatora "`==`" i "`!=`", *pod uvjetom da su elementi oba vektora istog tipa*. Pod tim uvjetom vektore čak možemo porediti i pomoću operatora "`<`", "`<=`", "`>`" i "`>=`", pri čemu se tada poređenje vrši koristeći *leksikografski kriterij* (tj. prema prvom po redu elementu koji se razlikuje u dva vektora). U slučaju nizova, ponovo bi se poredile *adrese*, što teško da može biti od osobite koristi.

Veoma interesantna mogućnost vektora je mogućnost dodavanja novih elemenata *na kraj* vektora, pri čemu se broj elemenata vektora pri takvom dodavanju *povećava za jedan*. To se postiže izvršavanjem funkcije "`push_back`" nad vektorom, pri čemu se kao parametar zadaje element koji se dodaje. Na primjer, naredba

```
ocjene.push_back(9);
```

povećava broj elemenata vektora "`ocjene`" za 1, i novododanom elementu (koji se nalazi *na kraju* vektora) dodjeljuje vrijednost "`9`". Kao što je već rečeno, sasvim je moguće deklarirati *prazan vektor*, odnosno vektor koji ne sadrži *niti jedan element*, a zatim operacijom "`push_back`" dodati onoliko elemenata u vektor koliko nam je potrebno. Ova strategija je naročito praktična u slučaju kada ne znamo unaprijed koliko će vektor imati elemenata (npr. kada elemente vektora unosimo s tastature, pri čemu se unos vrši sve dok na neki način ne signaliziramo da smo završili s unosom). Trenutno aktuelni broj elemenata u svakom trenutku možemo saznati pozivom funkcije "`size`" bez parametara nad vektorom. Razmotrimo, na primjer, sljedeći programski isječak:

```
std::vector<int> brojevi;
std::cout << "Unesite slijed brojeva, pri čemu 0 označava kraj unosa: ";
int broj;
do {
  std::cin >> broj;
  if(broj != 0) brojevi.push_back(broj);
} while(broj != 0);
std::cout << "Ovaj niz brojeva u obrnutom poretku glasi:\n";
for(int i = brojevi.size() - 1; i >= 0; i--) std::cout << brojevi[i] << " ";
```

U ovom primjeru, unosimo slijed brojeva u vektor, sve dok se ne unese nula. Pri tome je odgovarajući vektor na početku prazan, a svaki novouneseni broj (osim terminalne nule) dodajemo na kraj vektora pomoću funkcije "`push_back`". Na kraju, unesene brojeve ispisujemo u obrnutom poretku, pri čemu broj unesenih elemenata saznajemo pomoću funkcije "`size`".

Treba napomenuti da operator "**`sizeof`**" primijenjen na promjenljive vektorskog tipa daje rezultat koji *nije u skladu s intuicijom*. Zbog toga, razni prljavi trikovi sa "**`sizeof`**" operatorom, koji se u izvjesnim situacijama mogu iskoristiti za određivanje broja elemenata nekog niza, a koji se mogu susresti u nekim C programima (poput konstrukcije "**`sizeof`**`(niz)/`**`sizeof`**`(niz[0])`"), *ne daju ispravan rezultat* ukoliko se primijene na vektore (stvar je u tome što operator "**`sizeof`**" daje tzv. *interno zauzeće memorije* koje zauzima neki objekat, a ne *ukupno zauzeće*, koje može biti znatno veće) . Stoga, za određivanje broja elemenata vektora uvijek treba koristiti funkciju "`size`".

Veliki nedostatak nizova je u tome što se oni ne mogu prenositi kao parametri u funkcije. Doduše, zahvaljujući automatskoj pretvorbi nizova u pokazivače izgleda *kao da se nizovi mogu prenositi kao parametri u funkcije*, ali se pri tome u funkciju *ne prenosi niz*, već samo *pokazivač na prvi element niza*, a funkcija se ponaša *kao da operira s nizom* zahvaljujući činjenici da se na pokazivače može primjenjivati indeksiranje. Ipak, funkcija ni na kakav način ne može saznati broj elemenata niza koji joj je tobože "prenesen" kao parametar, nego se broj elemenata niza mora prenositi u funkciju kao *dodatni parametar*. Neka je, na primjer, potrebno napisati funkciju "`Prosjek`" koja kao rezultat vraća aritmetičku sredinu niza cijelih brojeva koji joj je "prenesen" kao parametar. Pošto nema načina da funkcija sazna koliko niz ima elemenata, broj elemenata se mora prenijeti kao dodatni parametar. Takva funkcija bi mogla izgledati recimo ovako:

```
double Prosjek(int niz[], int broj_elemenata) {
  double suma = 0;
  for(int i = 0; i < broj_elemenata; i++) suma += niz[i];
  return suma / broj_elemenata;
}
```

Ukoliko bismo ovu funkciju htjeli upotrijebiti da nađemo prosjek svih ocjena iz niza "`ocjene`", uz pretpostavku da ih ima 10, to bismo mogli uraditi ovako:

```
std::cout << "Prosjek ocjena iznosi " << Prosjek(ocjene, 10);
```

Vidimo da moramo eksplicitno prenositi broj ocjena kao parametar. Pored toga, formalni parametar "`niz`" unutar funkcije "`Prosjek`" *uopće nije niz*, *već pokazivač* (tako da bismo identičan efekat dobili kada bismo deklaraciju "`int niz[]`" zamijenili s "`int *niz`", što se u C programima često radi), a ponaša se slično poput niza isključivo zahvaljujući činjenici da se na pokazivače može primijeniti indeksiranje. S druge strane vektori se *zaista* mogu prenositi kao parametri u funkcije (bez konverzije u pokazivače), pri čemu je broj elemenata prenesenog vektora moguće saznati pomoću funkcije "`size`". Razmotrimo sljedeću funkciju, koja je analogna prethodnoj, ali radi s vektorima:

```
double Prosjek(std::vector<int> v) {
  double suma = 0;
  for(int i = 0; i < v.size(); i++) suma += v[i];
  return suma / v.size();
}
```

Primijetimo kako se deklarira formalni parametar funkcije vektorskog tipa – bez ikakve posebne specifikacije broja elemenata. Ukoliko je sada "`ocjene`" vektor, prosječnu ocjenu možemo naći sljedećim pozivom, bez obzira na to koliko ih ima:

```
std::cout << "Prosjek ocjena iznosi " << Prosjek(ocjene);
```

Kada budemo govorili o referencama, vidjećemo da se, radi nekih razloga vezanih za efikasnost, formalni parametri funkcija vektorskog tipa obično deklariraju na nešto drugačiji način, ali to ne mijenja smisao onoga o čemu na ovom mjestu govorimo.

Od verzije C++17, funkcija "`size`" se također može koristiti i sa sintaksom u kojoj se vektor šalje kao argument funkcije, umjesto da se funkcija primjenjuje na vektor (ta verzija funkcije "`size`" nalazi se u imeniku "`std`", tako da ga je potrebno navoditi ukoliko nismo koristili "`using`" deklaraciju. Tako se, od verzije C++17 nadalje, umjesto "`brojevi.size()`" može pisati i "`std::size(brojevi)`". Međutim, ova verzija funkcije "`size`" radi i s nizovima, a ne samo s vektorima (ali samo s *pravim nizovima*, a ne s "lažnjacima" koji se dobijaju kada niz "pošaljemo" funkciji, a koji su zapravo pokazivači, s obzirom da pokazivač ne sadrži nikakvu informaciju o tome koliko je velik niz na čiji početak oni pokazuju). Naime, zbog nekih tehničkih razloga, verzija funkcije "`size`" koja se poziva nad objektom, nije se mogla podržati tako da radi i s nizovima, ali zato ova druga funkcija može. Stoga, ta druga varijanta može olakšati pisanje univerzalnih funkcija koje mogu raditi s različitim vrstama podataka, recimo i s nizovima i s vektorima (u kasnijim predavanjima ćemo govoriti o tome kako se prave takve funkcije).

Kao argumenti funkcijama koje kao parametre očekuju vektore, mogu se također prosljeđivati i *liste inicijalizatora*, koje se tada interpretiraju kao vektori s odgovarajućim brojem elemenata i sadržajem koji odgovara listi inicijalizatora. Na primjer, ukoliko nas zanima aritmetička sredina (prosjek) brojeva 3, 4, 6, 2, 7, za tu svrhu možemo iskoristiti gore napisanu funkciju "`Prosjek`" na sljedeći način:

```
std::cout << Prosjek({3, 4, 6, 2, 7});
```

Ovo je pogodno mjesto da se razmotri još jedna korisna stvar u jeziku C++, a to je *rasponska for-petlja*. Ova vrsta petlje poznata je u mnogim drugim jezicima (u nekima pod nazivom *foreach petlja* jer se u njima ona realizira pomoću posebne ključne riječi "`foreach`"), a njena sintaksa u jeziku C++ je

for(*promjenljiva* **:** *kolekcija*) *tijelo_petlje*

Ovdje je "*promjenljiva*" neka promjenljiva, bilo neka deklarirana od ranije, bilo lokalno deklarirana u petlji (kao u klasičnoj *for*-petlji), "*kolekcija*" je neka kolekcija podataka koja može biti neki niz, vektor ili bilo koja druga kolekcija koju jezik C++ poznaje (pored nizova i vektora, jezik C++ posjeduje još mnogo tipova koji predstavljaju razne kolekcije podataka, od kojih ćemo neke upoznati kasnije), dok je "*tijelo_petlje*" naredba ili skupina naredbi koja sačinjava tijelo petlje, slično kao kod svih drugih tipova petlji. Tip promjenljive "*promjenljiva*" mora se slagati s tipom podataka pohranjenih u kolekciji "*kolekcija*". Smisao ove petlje je da će se tijelo petlje izvršiti onoliko puta koliko ima elemenata u kolekciji, pri čemu će u svakom prolasku kroz petlju promjenljiva "*promjenljiva*" dobijati vrijednost jednog po jednog elementa kolekcije, redom od prvog do posljednjeg (tačnije, u tu promjenljivu će se *kopirati* odgovarajući elementi kolekcije, jedan po jedan). Recimo, ukoliko želimo ispisati sve vrijednosti vektora "`v`" čiji su elementi realni brojevi na ekran (svaki element u posebnom redu), pomoću obične *for*-petlje to bismo mogli uraditi ovako:

```
for(int i = 0; i < v.size(); i++) std::cout << v[i] << std::endl;
```

S druge strane, pomoću rasponske *for*-petlje istu stvar možemo jednostavnije izvesti ovako:

```
for(double x : v) std::cout << x << std::endl;        // Rasponska for-petlja...
```

Treba primijetiti da kod rasponske *for*-petlje ne postoji pojam brojača petlje, za razliku od klasičnih *for*-petlji kod kojih je pojam brojača petlje od ključne važnosti. Također, unutar rasponske *for*-petlje nad kolekcijom kroz koju prolazimo *nije dozvoljeno izvršiti nikakvu operaciju koja mijenja broj elemenata te kolekcije* (kao što je "`resize`" ili "`push_back`"). U suprotnom će petlja imati *nepredvidljivo ponašanje*.

Kao još jednu ilustraciju rasponske *for*-petlje napisaćemo i verziju funkcije "`Prosjek`" koja računa aritmetičku sredinu elemenata vektora, ali ovaj put pomoću rasponske *for*-petlje:

```
double Prosjek(std::vector<int> v) {
  double suma = 0;
  for(int x : v) suma += x;                           // Rasponska for-petlja...
  return suma / v.size();
}
```

Rasponska *for*-petlja može se primijeniti i direktno na liste inicijalizatora. Stoga je konstrukcija poput sljedeće posve legalna (ispisuje brojeve 3, 5, 2, 8 i 6 na ekran, svaki u novom redu):

```
for(int x : {3, 5, 2, 8, 6}) std::cout << x << std::endl;
```

Rasponska petlja radi i s nizovima, ali samo s *pravim nizovima*, ne i s lažnim "nizovima" koji su formalni parametri funkcija koje tobože primaju nizove kao parametre (slično varijanti funkcije "`size`" uvedenoj u C++17 koja prima vektor ili niz kao argument), jer su oni zapravo pokazivači, a ne nizovi, kao što smo maločas pojasnili. Stoga se prethodno napisana verzija funkcije "`Prosjek`" koja računa aritmetičku sredinu elemenata niza *ne može napisati* pomoću rasponske *for*-petlje, jer njen formalni parametar "`niz`" nije pravi niz, već pokazivač. Razlog za ovo ograničenje leži u tome što rasponska *for*-petlja mora znati koliko kolekcija posjeduje elemenata, a iz samog pokazivača (koji je obična adresa) nije moguće izvući takvu informaciju.

Nizovi se ne mogu vraćati kao rezultati iz funkcija, što je često veliko ograničenje, koje se može zaobići samo pomoću nekih dosta nezgrapnih konstrukcija, koje uključuju prenošenje adrese odredišnog niza koji bi trebao da prihvati rezultat kao dodatni parametar u funkciju. Sve ove zavrzlame nisu potrebne ukoliko koristimo vektore, jer se vektori najnormalnije mogu vraćati kao rezultati iz funkcije. Razmotrimo, na primjer, sljedeću funkciju, koja sabira dva cjelobrojna vektora koja su joj prenesena kao parametri, i vraća kao rezultat zbir ta dva vektora računat element po element. Radi jednostavnosti, pretpostavimo da vektori koji se prenose kao parametri u ovu funkciju imaju jednak broj elemenata (da izbjegnemo nedoumice šta tačno treba raditi ukoliko to nije ispunjeno):

```
std::vector<int> ZbirVektora(std::vector<int> a, std::vector<int> b) {
  std::vector<int> c(a.size());
  for(int i = 0; i < a.size(); i++) c[i] = a[i] + b[i];
  return c;
}
```

Istu stvar smo mogli uraditi i ovako, uz upotrebu funkcije "`push_back`":

```
std::vector<int> ZbirVektora(std::vector<int> a, std::vector<int> b) {
  std::vector<int> c;
  for(int i = 0; i < a.size(); i++) c.push_back(a[i] + b[i]);
  return c;
}
```

Bez obzira koje smo od ova dva rješenja primijenili, ukoliko su "`v1`", "`v2`" i "`v3`" tri cjelobrojna vektora, možemo koristiti naredbu poput

```
v3 = ZbirVektora(v1, v2);
```

da saberemo vektore "`v1`" i "`v2`" i rezultat smjestimo u vektor "`v3`". Ništa nalik ovome nije moguće ukoliko su "`v1`", "`v2`" i "`v3`" nizovi, a ne vektori. Nije na odmet napomenuti i da kod funkcija koje kao rezultat vraćaju vektore, iza naredbe "`return`" može se navesti lista inicijalizatora, koja će se onda pretvoriti u odgovarajući vektor, koji će biti vraćen kao rezultat iz funkcije.

Pretpostavimo sada da smo od ranije imali deklarirana dva vektora "`v1`" i "`v2`" i da želimo da kreiramo *novi vektor* "`v3`" (koji nije postojao od ranije) i da ga *inicijaliziramo* na zbir vektora "`v1`" i "`v2`". Klasični načini da to izvedemo su sljedeći (pri čemu prvi koristi kopirajuću a drugi konstruktorsku sintaksu za inicijalizaciju), u kojima je iskorištena osobina da je legalno inicijalizirati jedan vektor drugim vektorom:

```
std::vector<int> v3 = ZbirVektora(v1, v2);                    // Kopirajuća sintaksa
std::vector<int> v3(ZbirVektora(v1, v2));                     // Konstruktorska sintaksa
```

Ove konstrukcije su sasvim logične, ali su prilično nezgrapne. Stoga je u jezik C++ uvedena još jedna interesantna olakšica koja se naziva *automatsko određivanje tipa promjenljive*. Naime, primijetimo da je s logičkog aspekta čisti višak navoditi da promjenljiva "`v3`" ima tip "`std::vector<int>`", s obzirom da je jasno da ona *mora imati taj tip* na osnovu toga što povratni tip funkcije "`ZbirVektora`" upravo takav. Tako je dopušteno da se umjesto prethodnih konstrukcija piše prosto

```
auto v3 = ZbirVektora(v1, v2);                                // Kopirajuća sintaksa
auto v3(ZbirVektora(v1, v2));                                 // Konstruktorska sintaksa
```

Ključna riječ "`auto`" označava da se tip promjenljive *automatski određuje na osnovu tipa onoga čime se ona inicijalizira* (stoga je jasno da se "`auto`" može koristiti samo kod promjenljivih koje se inicijaliziraju odmah pri stvaranju). Automatsko određivanje tipa radi sa svim tipovima podataka, a ne samo s komplikovanijim tipovima podataka kao što su vektori. Stoga su sljedeće deklaracije sasvim legalne i nakon njih će promjenljiva "`a`" imati tip "`int`", a promjenljiva "`b`" tip "`double`":

```
auto a = 3;                                                   // Može i "auto a(3);"
auto b = 2.15;                                                // Može i "auto b(2.15);"
```

Ipak, sve promjenljive koje se deklariraju unutar jedne "`auto`" deklaracije moraju imati *konzistentan tip*, odnosno za svaku od njih se mora donijeti *isti zaključak o tome kakav bi njihov tip trebao biti*. Stoga je sljedeća deklaracija *neispravna*, jer se za promjenljive "`a`" i "`b`" donosi *različit zaključak* o njihovom tipu:

```
auto a = 3, b = 2.15;                                         // Ovo je NEISPRAVNO!
```

Nažalost, zbog jedne nesmotrene odluke autora jezika C++, "`auto`" deklaracije se u standardima starijim od C++17 *praktično ne mogu koristiti s jednoobraznom inicijalizacijom* (tj. inicijalizacijom pomoću vitičastih zagrada). Naime, sve do standarda C++17, vrijedila je nesretno odabrana konvencija da ukoliko se u "`auto`" deklaraciji pojave vitičaste zagrade, odgovarajuća promjenljiva *obavezno dobija tip liste inicijalizatora*. Recimo, u deklaraciji

```
auto a{3};              // Do C++17, tip od "a" nije "int" nego lista inicijalizatora!
```

promjenljiva "`a`" neće biti tipa "`int`" (inicijalizirana na vrijednost 3), nego tipa liste inicijalizatora (inicijalizirana da predstavlja listu inicijalizatora s jedni, elementom čija je vrijednost 3). Mnogi korisnici jezika C++ izrazili su veliko nezadovoljstvo ovakvom odlukom, tako da je u standardu C++17 jezika C++ uvedena izmjena po kojoj će prethodni primjer ipak zaključiti da je "`a`" tipa "`int`" (za razliku od konstrukcije "`auto a = {3};`" koja i dalje deklarira promjenljivu "`a`" tipa liste inicijalizatora).

Deklaracije s automatskim određivanjem tipa su nesumnjivo veoma korisne, pogotovo kada je tip promjenljive rogobatan, ali s njima *ne treba pretjerivati*. Ukoliko je tip promjenljive jednostavan, uvijek je pametnije *eksplicitno naznačiti njen tip*, nego oslanjati se na automatsko određivanje tipa.

Treba obratiti pažnju na još jedan detalj vezan za automatsko određivanje tipa koji može biti zbunjujući. Naime, kod automatskog određivanja tipa, *nizovni tipovi se degradiraju u pokazivačke tipove*, odnosno *nizovi se tretiraju kao pokazivači*. Tako, u sljedećem primjeru, mada je "`a`" nizovnog tipa (tačnije, tipa niza od 10 cijelih brojeva), "`b`" nije nizovnog tipa, nego tipa *pokazivača na cijele brojeve* (koji se, usput, inicijalizira da pokazuje na prvi element niza "`a`"):

```
int a[10];                          // "a" je tipa niza od 10 elemenata tipa "int"
auto b = a;                         // Tip od "b" je pokazivač na objekte tipa "int"
```

Vidjeli smo da je vektore moguće prenositi kao parametre u funkcije i vraćati ih kao rezultate iz funkcija. S vektorima je čak moguće otići i korak dalje. Naime, izrazi poput "`v1 + v2`" nemaju smisla za nizove, a podrazumijevano nemaju smisla ni za vektore. Međutim, u slučaju vektora, moguće je definirati smisao svih operatora čije značenje nije inicijalno definirano. Na primjer, dovoljno je da u ranije datom primjeru funkciji "`ZbirVektora`" promijenimo ime u "`operator +`" pa da operator "`+`" postane definiran nad vektorima. Drugim riječima, ukoliko napišemo

```
std::vector<int> operator +(std::vector<int> a, std::vector<int> b) {
  std::vector<int> c;
  for(int i = 0; i < a.size(); i++) c.push_back(a[i] + b[i]);
  return c;
}
```

tada ćemo za sabiranje dva vektora moći koristiti sintaksu poput "`v3 = v1 + v2`", odnosno zbir vektora "`v1`" i "`v2`" će se računati pozivanjem funkcije "`operator +(v1, v2)`". Ovaj primjer je naveden ovdje samo kao ilustracija mogućnosti koju posjeduju vektori i njihovoj povećanoj fleksibilnosti u odnosu na nizove. Za potpuno razumijevanje ovog primjera potrebno je upoznati se s pojmom *operatorskih funkcija*, o čemu ćemo detaljno govoriti kada budemo govorili o preklapanju (preopterećivanju) operatora.

Kada koristimo nizove, nije jednostavno umetnuti novi element između dva proizvoljna elementa niza, niti izbrisati element koji se nalazi na proizvoljnoj poziciji u nizu. Naime, umetanje novog elementa između dva elementa zahtijeva pomjeranje svih elemenata niza koji slijede iza novoumetnutog elementa za jedno mjesto naviše (što možemo izvesti recimo *for*-petljom), dok brisanje elementa na proizvoljnoj poziciji u nizu zahtijeva pomjeranje svih elemenata koji slijede iza elementa koji se briše za jedno mjesto naniže. U slučaju vektora, za umetanje odnosno brisanje elemenata na proizvoljnoj poziciji na raspolaganju su funkcije "`insert`" odnosno "`erase`". Da bismo u vektor "`v`" na poziciju "`n`" umetnuli element "`x`" (istiskujući naviše sve elemente počev od pozicije "`n`" nadalje) koristimo konstrukciju

```
v.insert(v.begin() + n, x);
```

dok za brisanje elementa na poziciji "`n`" (uz odgovarajuće pomjeranje naniže elemenata koji slijede) koristimo konstrukciju

```
v.erase(v.begin() + n);
```

Pri tome se veličina vektora automatski povećava (pri umetanju) odnosno smanjuje (pri brisanju) za jedinicu. Moguće je čak i obrisati odjednom sve elemente od pozicije "`n1`" do pozicije prije "`n2`" (tj. ne računajući element tačno na poziciji "`n2`") konstrukcijom poput

```
v.erase(v.begin() + n1, v.begin() + n2);
```

Puni smisao ovih konstrukcija biće jasan kada se upoznamo s tzv. *iteratorima*, jer do tada ne možemo objasniti šta predstavlja funkcija "`begin`" koja se javlja u ovim konstrukcijama (inače, funkcije "`insert`" i "`erase`" se mogu koristiti na još mnogo različitih načina, ali u detalje ne možemo ulaziti). Također, treba znati da funkcije "`insert`" i "`erase`" vrše premještanje elemenata vektora u memoriji (isto kao da to radimo ručno *for*-petljom), tako da se za slučaj velikih vektora one *ne izvode efikasno*.

Bitno je naglasiti da se prilikom korištenja indeksiranja ni kod nizova ni kod vektora ne provjerava da li se indeks prilikom pristupa nekom elementu nalazi u dozvoljenom opsegu. To, na primjer znači da ukoliko niz ili vektor "`a`" ima recimo 5 elemenata, sljedeća naredba će "izbombardovati" memoriju bez obzira da li je "`a`" niz ili vektor:

```
for(int i = 0; i < 30000; i++) a[i] = 1000;              // Ovo ne smijemo raditi!
```

Posljedice ovakvog "bombardiranja" su nepredvidljive. Još je najbolja varijanta ukoliko operativni sistem primijeti "zločesto" ponašanje programa pa ga "ubije" (uz prijavu poruke poput "This program performed illegal operation and will be shut down" ili neke slične). Međutim, postoji mogućnost da program "izbombarduje" sam sebe (ili svoje vlastite promjenljive), što operativni sistem ne može detektirati. Takve akcije mogu dugo ostati neprimijećene, odnosno mogu voditi ka programima koji počinju da rade neispravno tek nakon dužeg vremena. Na ovakve greške treba dobro paziti, jer se obično teško otkrivaju. Što je još gore, prema standardu jezika C i C++, bilo kakav pristup elementu putem ilegalnog indeksa smatra se *nedefiniranim ponašanjem*, čak i kad se preko ilegalnog indeksa vrši *samo čitanje*. Mnogi očekuju da je najgore što se može desiti u slučaju čitanja putem ilegalnog indeksa da će biti pročitana neka besmislena vrijednost, ali zaista se može desiti bilo šta (čak i na nekom mjestu u programu koje nema direktne veze s mjestom na kojem je došlo do ilegalnog pristupa), pogotovo kod kompajlera koji vrše agresivnu optimizaciju. Recimo, na mnogim kompajlerima, sljedeći programski isječak (u kojem se vrši nelegalni pristup nepostojećem elementu niza s indeksom 5) petlja se pretvara u beskonačnu ako je pri kompajliranju korištena agresivna optimizacija, jer u fazi optimizacije kompajler zaključi da čim se "`i`" koristi za indeksiranje, a posljednji validni indeks je 4, "`i`" ne može biti veći od 5 čak i nakon povećanja za 1 koje obavlja petlja. Time kompajler zaključuje da uvjet "`i <= 5`" mora biti uvijek tačan, i odlučuje da uopće ne generira kod za njegovu provjeru, nego ga prosto zamijeni s konstantom "`true`". Posljedica je da petlja postaje beskonačna (vrlo malo ljudi je svjesno ovoga):

```
int niz[5] = {10, 20, 30, 40, 50};
for(int i = 0; i <= 5; i++) std::cout << niz[i] << std::endl;  // Treba "<" a ne "<="
```

Ovo je zaista velika opasnost. Stoga, da bi se omogućila bolja kontrola, kod vektorskih tipova je uvedena i funkcija "`at`" koja obavlja sličnu ulogu kao i uglaste zagrade, ali uz kontrolu ispravnosti indeksa. Preciznije, izrazi oblika "`a[i]`" i "`a.at(i)`" su u potpunosti identični, ali se u drugom slučaju provjerava da li je indeks u dozvoljenom opsegu. Ukoliko nije, dolazi do bacanja izuzetka. Šta su izuzeci, i na koji način se mogu "uhvatiti" bačeni izuzeci, govorićemo kasnije. Uglavnom, neuhvaćeni izuzetak trenutno dovodi do prekida programa. Tako, ukoliko je "`a`" vektor od 5 elemenata, prilikom izvršavanja naredbe

```
for(int i = 0; i < 30000; i++) a.at(i) = 1000;            // Ovo je sigurnije...
```

doći će do bacanja izuzetka čim promjenljiva "`i`" dostigne vrijednost 5. Uz pretpostavku da izuzetak nije uhvaćen, to će odmah dovesti do prekida programa, prije nego što program uspije napraviti neku drugu štetu. Stoga, funkciju "`at`" treba koristiti *kad god nam je sigurnost bitna* (početnicima se savjetuje da je koriste *uvijek*, tj. *bezuvjetno*). S druge strane, upotreba uglastih zagrada za indeksiranje je *efikasnija*, s obzirom da se kod funkcije "`at`" gubi izvjesno vrijeme na kontrolu ispravnosti indeksa, tako da uglaste zagrade treba koristiti u slučaju da nam je *efikasnost jako bitna* (i *samo tada*), pri čemu smo *potpuno sigurni šta radimo*. U svakom slučaju, ostavljena nam je mogućnost izbora.

Elementi vektora mogu biti ma kojeg legalnog tipa (vidjećemo uskoro da elementi vektora mogu i sami biti ponovo vektori). Recimo, ukoliko želimo deklarirati vektor čiji su elementi kompleksni brojevi, izvedeni iz tipa "**`double`**", odgovarajuća deklaracija glasi ovako:

```
std::vector<std::complex<double>> v;
```

Ranije smo naveli da promjenljive tipa "**`bool`**", suprotno očekivanjima, zauzimaju jedan bajt, a ne jedan bit u memoriji. Stoga niz od recimo 10000 elemenata tipa "**`bool`**" zauzima čitavih 10000 bajta. Međutim, pri upotrebi vektora izbjegnuto je ovo rasipanje memorije, tako da unutar vektora jedan element tipa "**`bool`**" zauzima *svega jedan bit* (standard jezika C++ doduše ne propisuje, nego samo preporučuje autorima kompajlera da izvrše ovu optimizaciju, ali nju koriste praktično svi raspoloživi kompajleri). Drugim riječima, vektor od 10000 elemenata tipa "**`bool`**" tipično zauzima svega 10000 bita, osnosno 1250 bajta (8 puta manje). Nažalost, ova kompresija ima svoju cijenu. Na prvom mjestu, pristup elementima vektora čiji su elementi tipa "**`bool`**" značajno je sporiji nego pristup elementima nizova čiji su elementi tipa "**`bool`**" (ili elementima vektora čiji su elementi recimo tipa "**`int`**"). Ovo je tipični primjer pojave koja je u računarstvu poznata kao *trgovanje između prostora i vremena* (engl. *space-time tradeoff*) prema kojoj se ušteda u zauzeću memorije vrlo često plaća gubitkom po pitanju vremena izvršavanja i obrnuto. Drugi problem je što ova kompresija dovodi do "pakovanja" više elemenata vektora (tipično 8) u jednu memorijsku ćeliju, tj. na istu adresu, tako da postaje besmisleno govoriti o adresi pojedinih elemenata vektora. Stoga nam kompajler neće dozvoliti da uzmemo adresu nekog elementa vektora čiji su elementi tipa "**`bool`**" i da je dodijelimo nekom pokazivaču na objekte tipa "**`bool`**". To je razlog zbog kojeg primjer s pokazivačkom aritmetikom koji smo ranije demonstrirali neće raditi s vektorima čiji su elementi tipa "**`bool`**".

Nije na odmet spomenuti i još nekoliko jednostavnih funkcija bez parametara koje podržavaju vektori i koje se pozivaju nad vektorima. Funkcije "`front`" i "`back`" vraćaju prvi odnosno posljednji element vektora respektivno, tako da su "`v.front()`" odnosno "`v.back()`" sinonimi za "`v[0]`" odnosno "`v[v.size() - 1]`". Funkcija "`data`" vraća pokazivač na prvi element vektora, tako da je "`v.data()`" sinonim za "`&v[0]`" (osim za vektore čiji su elementi tipa "**`bool`**", za koje ova funkcija nije podržana). Funkcija "`pop_back`" uklanja posljednji element vektora uz smanjenje njegove veličine za 1, tako da je "`v.pop_back()`" sinonim za konstrukciju "`v.resize(v.size() - 1)`". Funkcija "`empty()`" vraća informaciju da li je vektor prazan ili ne, tako da je "`v.empty()`" sinonim za test "`v.size() == 0`". Funkcija "`clear`" uklanja sve elemente vektora (nakon čega vektor ostaje prazan), tako da je "`v.clear()`" sinonim za "`v.resize(0)`". Interesantna je i funkcija "`reserve`", koja za razliku od ostalih navedenih funkcija, ima i parametar. Ona rezervira prostor u memoriji za eventualna proširenja veličine vektora do vrijednosti navedene parametrom (kao npr. u konstrukciji "`v.reserve(1000)`"), što omogućava da će se proširenja koja vrše funkcije "`resize`", "`push_back`" i "`insert`" sve do rezervirane veličine vršiti daleko efikasnije (bez raznih "pretumbavanja" po memoriji i sličnih aktivnosti) nego u slučaju kada prostor nije unaprijed rezerviran. Funkcija "`capacity`" bez parametara pozvana nad vektorom vraća količinu rezerviranog prostora (izraženu u broju elemenata koji se mogu smjestiti u rezervirani prostor), koja je u svakom slučaju uvijek veća ili jednaka od trenutnog broja elemenata vektora. Konačno, funkcija "`shrink_to_fit`" pozvana nad vektorom (bez parametara) uklanja sav dodatno rezervirani prostor, čime se postiže da vektor ne rezervira ništa veći prostor u memoriji nego što je potrebno za smještaj njegovih elemenata. Ovo je zgodno kada nam je potrbna ušteda memorije.

Sljedeća osobina jezika C++ je u prvi mah nešto teža za razumijevanje i početnici će je vjerovatno malo (ili nimalo) koristiti. Međutim, kako se ova osobina smatra revolucionarnom i u stanju je da dramatično poveća brzinu i efikasnost C++ programa, zaslužuje da se o njoj na ovom mjestu barem ponešto kaže. U pitanju je takozvana *semantika pomjeranja* (neki je nazivaju i *semantika prenosa*) ili *move-semantika*. O čemu se radi? Uzmimo na primjer da su "`v1`" i "`v2`" dva vektora s istim tipom elemenata. Izvršimo li dodjelu poput "`v1 = v2`", svi elementi vektora "`v2`" će se kopirati u elemente vektora "`v1`" (nakon prilagođavanja njegove veličine), odnosno nakon ove dodjele, vektori "`v1`" i "`v2`" će imati identičan sadržaj (bez obzira što su to *dva različita objekta*, tj. nalaze se na *različitim mjestima u memoriji*). Problem je što ovo kopiranje elemenata može dugo trajati (ukoliko vektor "`v2`" ima 1000000 elemenata, potrebno je iskopirati upravo toliko elemenata). S druge strane, nekada nam je samo potrebno da se sadržaj vektora "`v2`" prosto pojavi u vektoru "`v1`", a da nas pri tome nije briga šta će se poslije desiti s vektorom "`v2`". Stoga, ukoliko umjesto dodjele "`v1 = v2`" izvršimo nešto poput

```
v1 = std::move(v2);                    // Semantika pomjeranja...
```

elementi vektora "`v2`" će se prosto "preseliti" u vektor "`v1`", dok će vektor "`v2`" ostati *prazan* (tj. bez elemenata). Šta se ovim postiže? Poenta je što se ova "selidba" može tehnički izvesti *bez ikakvog kopiranja elemenata* (intuitivno, vektor "`v1`" se prosto "useli" u onaj dio memorije gdje se prije nalazio vektor "`v2`", dok se sami elementi ne pomjeraju u memoriji), tako da je ova konstrukcija neuporedivo efikasnija od dodjele "`v1 = v2`" (štaviše, njeno trajanje uopće ne ovisi od broja elemenata u vektoru "`v2`"). Kroz jedan primjer ćemo ilustrirati kada ovo može biti jako korisno. Pretpostavimo da želimo *razmijeniti* sadržaj dva vektora "`v1`" i "`v2`" (tj. želimo da se elementi vektora "`v1`" pojave unutar vektora "`v2`", a elementi vektora "`v2`" unutar vektora "`v1`"). Klasično bismo to uradili na isti način kao kad želimo razmijeniti recimo dvije cjelobrojne promjenljive. Uvešćemo pomoćni vektor (tj. pomoćnu promjenljivu vektorskog tipa) nazvan recimo "`pomocni`", a zatim ćemo izvršiti sekvencu naredbi

```
pomocni = v1; v1 = v2; v2 = pomocni;
```

Ova konstrukcija radi, ali je problem što se u njoj javljaju tri kopiranja vektora. Ako vektori "`v1`" i "`v2`" imaju po 1000000 elemenata, to je ukupno 3000000 kopiranja individualnih elemenata! Da li su ovolika kopiranja zaista neophodna da bismo razmijenili dva vektora? Ako malo bolje razmislimo, vidjećemo da bismo sva kopiranja mogli izbjeći kad bismo mogli nekako postići da se vektor "`v1`" prosto "useli" u onaj dio memorije gdje se prije nalazio vektor "`v2`", a vektor "`v2`" u onaj dio memorije gdje se prije nalazio vektor "`v1`". Koristeći semantiku pomjeranja, to možemo postići recimo ovako:

```
pomocni = std::move(v1); v1 = std::move(v2); v2 = std::move(pomocni);
```

Ovdje se prvo vektor "`pomocni`" "useljava" u onaj dio memorije gdje se nalazio vektor "`v1`" (koji pri tome biva uništen). Zatim se vektor "`v1`" "useljava" u onaj dio memorije gdje se nalazio vektor "`v2`" i na kraju se vektor "`v2`" "useljava" u onaj dio memorije gdje se nalazio vektor "`pomocni`", odnosno gdje se na početku nalazio vektor "`v1`". Ovim je izvršena razmjena dva vektora, a da pri tome niti jedan element nije premještan u memoriji, što je neuporedivo efikasnije!

Nakon ovog ilustrativnog primjera, treba reći šta zapravo tačno radi funkcija "`move`" (s obzirom na sintaksu upotrebe, može se naslutiti da je "`move`" zaista neka vrsta funkcije). Ova funkcija zapravo daje kao rezultat *isti objekat na koji je primijenjena*, samo *doveden u jedno specijalno stanje*. To specijalno stanje naziva se *x-vrijednost* (engl. *x-value*), pri čemu ovo "*x*" potiče od engl. *expired*. Ono signalizira da nas *nije briga šta će se s tim objektom desiti kasnije*, odnosno da nikakva šteta nije ni *ako on bude uništen*. Dakle, ako je "`v`" recimo neki vektor, "`std::move(v)`" daje kao rezultat upravo taj isti vektor "`v`", ali doveden u specijalno stanje koje signalizira da nas *nije briga* šta će se s tim vektorom kasnije desiti. Operator dodjele "`=`" za neke tipove podataka zna prepoznati to specijalno stanje, i ukoliko ga uoči, neće se ustručavati da dodjelu izvrši tako što će prosto zauzeti onaj prostor u memoriji koji je taj objekat zauzimao (inače, to specijalno stanje se može prepoznavati u još nekim situacijama, a ne samo prilikom izvođenja dodjela, recimo prilikom prenosa objekata kao parametara u funkcije ili prilikom vraćanja objekata kao rezultata iz funkcija). To objašnjava kako zapravo radi konstrukcija poput "`v1 = std::move(v2)`". Kao što vidimo, usprkos imenu, funkcija "`move`" zapravo *ništa ne pomjera*, nego samo označava da će objekat na koji je primijenjena prvom prilikom kada se zatraži njegovo kopiranje biti *pomjeren*, a ne zaista kopiran.

Ne podržavaju svi tipovi podataka *move*-semantiku. Ona je uglavnom podržana kod masivnijih tipova podataka koji zauzimaju veću količinu memorije. Na primjer, ona nije podržana za proste tipove podataka kao što je "`int`". U takvim slučajevima, specijalno stanje koje biva uspostavljeno pozivom funkcije "`move`" se prosto ignorira. Recimo, za cjelobrojne promjenljive "`a`" i "`b`", nema nikakve razlike između konstrukcija "`a = b`" i "`a = std::move(b)`". Ukoliko nam treba *move*-semantika, a nismo sigurni da li je

neki tip podataka podržava, nema ništa loše u tome da probamo koristiti funkciju "move". U slučaju da *move*-semantika nije podržana, neće se desiti ništa loše, osim što ćemo dobiti klasično kopiranje, odnosno neće biti nikakvog dobitka na efikasnosti u odnosu na klasični pristup. Treba još reći da se funkcija "move" izvorno nalazi u biblioteci "utility", ali se također nalazi i u raznim drugim bibliotekama kao što je "vector" (drugim riječima, ukoliko smo uključili zaglavlje biblioteke "vector" u program, nije potrebno uključivati i zaglavlje biblioteke "utility" da bismo mogli koristiti funkciju "move").

Svi dosada izloženi argumenti sasvim su dovoljni da ilustriraju prednosti vektora u odnosu na obične nizove, tako da se pri programiranju u C++-u savjetuje da se vektori koriste umjesto nizova gdje je god to moguće. Zapravo, upotreba nizova umjesto vektora u jeziku C++ prakticira se isključivo u situacijama kada se unaprijed zna tačan broj elemenata koje je potrebno pohraniti i kada se taj broj ne mijenja tokom izvršavanja programa, kao recimo u primjeru niza od 12 elemenata koji čuva broj dana u pojedinim mjesecima godine (jasno je da takav niz mora imati tačno 12 elemenata). Štaviše, čak se ni u takvim slučajevima ne preporučuje upotreba klasičnih nizova (odnosno, savjetuje se da se klasični C-ovski nizovi *potpuno napuste*), nego se preporučuje da se u takvim slučajevima koristi novi tip podataka "array". U nastavku ćemo se u najkraćim crtama osvrnuti i na ovaj tip podataka. U nedostatku boljeg imena, primjerke tipa "array" zvaćemo *moderni nizovi* (da bismo ih razlikovali od običnih nizova naslijeđenih iz jezika C).

Moderni nizovi, slično kao i klasični nizovi, imaju fiksan broj elemenata, i kod njih je broj elemenata svojstvo samog tipa, a ne svojstvo individualne promjenljive. Samim tim, broj elemenata modernih nizova ne može se mijenjati tokom njihovog života. Tip "array" je bibliotečki definirani tip, i za njegovu upotrebu potrebno je uključiti istoimenu biblioteku (tj. biblioteku "array"), kao i koristiti prefiks "std::", osim ukoliko upotrebu tog prefiksa zaobiđemo pomoću naredbe "**using**". Promjenljive ovog tipa deklariraju se pomoću konstrukcije

```
std::array<tip_elemenata, broj_elemenata> ime_promjenljive
```

pri čemu "*broj_elemenata*" mora biti *prava konstanta* (ne smije biti neprava konstanta, promjenljiva ili proizvoljan izraz). Opcionalno se nakon imena promjenljive, kao kod običnih nizova, može nalaziti lista inicijalizatora (sa ili bez znaka "=" ispred) koja omogućava inicijalizaciju elemenata (modernog) niza. Lista inicijalizatora pri tom ne smije sadržavati više elemenata nego što iznosi veličina niza, a ukoliko sadrži manje elemenata, nedostajući elementi se dopunjavaju podrazumijevanom vrijednošću za tip elemenata niza (koji je 0 za sve brojčane tipove). Ukoliko se lista inicijalizatora ne navede, elementi niza prosto ostaju *neinicijalizirani*, kao kod običnih nizova. Nikakav drugi način deklariranja modernih nizova nije dozvoljen. Nakon deklaracije, moderni nizovi koriste se manje-više kao i obični nizovi. Međutim, moderni nizovi posjeduju i dosta inovacija koje posjeduju vektori, a ne posjeduju obični nizovi. Recimo, pored indeksacije pomoću uglastih zagrada, oni *podržavaju i indeksaciju pomoću funkcije* "at" uz *provjeru legalnosti indeksa*. Za razliku od običnih nizova, oni se *mogu dodjeljivati jedan drugom* (pod uvjetom da su istih tipova) i *mogu im se dodjeljivati odgovarajuće liste inicijalizatora* (uz uvjet da im broj elemenata nije veći od deklariranog broja elemenata u nizu). Broj elemenata modernog niza može se, kao i kod vektora, saznati primjenom funkcije "size" (samo što je taj broj elemenata nepromjenljiv tokom života promjenljive). Moderni nizovi također podržavaju rasponske *for*-petlje, kao i poređenja korištenjem operatora "==", "<", itd. Slično kao i kod vektora, kod njih se također ne vrši automatska pretvorba imena modernog niza u pokazivač na prvi element niza, nego se to, u slučaju potrebe, treba obaviti ručno. Umjesto toga, ime modernog niza označava *taj niz kao cjelinu*. Pored toga što to omogućava međusobno dodjeljivanje modernih nizova, to omogućava i *prenošenje modernih nizova kao cjelina u funkcije*, na identičan način kao kod vektora (umjesto da se prenese samo adresa prvog elementa niza), pod uvjetom da se tipovi formalnog i stvarnog parametra slažu, kao i *vraćanje modernih nizova kao rezultata iz funkcija* (pri tome, kad navodimo da je parametar funkcije ili povratna vrijednost tip modernog niza, ne smijemo zaboraviti navesti i broj elemenata, koji je sastavni dio samog tipa). Isto tako, kao i kod vektora, funkcije koje kao formalne parametre primaju moderne nizove, prihvatiće da im se kao stvarni parametri pošalju i *odgovarajuće liste inicijalizatora* (prikladne veličine i tipa elemenata). Neke od navedenih svojstava modernih nizova ilustrira sljedeći programski isječak:

```
std::array<int, 5> a;
std::array<int, 5> b{2, 3, 7, 5};
std::array<double, 4> c = {2.15, 3.4};
for(int i = 0; i < b.size(); i++) std::cout << b[i] << " ";      // 2 3 7 5 0
for(double x : c) std::cout << x << " ";                         // 2.15 3.4 0 0
a = b; a.at(4) = 1;
for(int x : a) std::cout << x << " ";                            // 2 3 7 5 1
b = {1, 3, 2};
for(int x : b) std::cout << x << " ";                            // 1 3 2 0 0
```

Treba naglasiti da kako je broj elemenata kod modernih nizova sastavni dio tipa, to konstrukcije poput "`array<int, 5>`" i "`array<int, 8>`" predstavljaju *dva posve različita tipa*, tako da promjenljive tih tipova *nisu međusobno saglasne* (npr. ne mogu se međusobno dodjeljivati ili porediti). Kako funkcije koje kao parametre "primaju" obične nizove neće primiti kao parametar moderne nizove (one zapravo očekuju pokazivač na prvi element niza, a ne sam niz), za tu svrhu treba koristiti funkcija "`data`" bez parametara (npr "`a.data()`"), koja daje pokazivač na prvi element niza, koji se može poslati kao parametar nekoj funkciji koja očekuje obični niz kao parametar (u skladu s onim što smo ranije rekli o funkciji "`data`", "`a.data()`" je zapravo isto što i "`&a[0]`").

S obzirom da su moderni nizovi, poput običnih nizova, uvijek iste veličine, operacije koje podržava vektor, a koje mijenjaju njihovu veličinu, nisu podržane u modernim nizovima. Tako, moderni nizovi ne podržavaju funkcije poput "`resize`", "`push_back`", "`insert`", "`erase`", "`pop_back`" i "`clear`" (također nisu podržane ni funkcije "`reserve`", "`capacity`" i "`shrink_to_fit`", jer za moderne nizove one gube smisao). Moderne nizove treba koristiti (umjesto vektora) kadgod radimo s kolekcijom elemenata čija je veličina *fiksna* i *unaprijed poznata*. Na taj način zadržavamo neke prednosti koji imaju vektori nad običnim nizovima, a ne opterećujemo program "komplikovanom mašinerijom" koju vektori "vuku za sobom", koja je neophodna za ostvarivanje svih fleksibilnosti koju oni posjeduju (vidjećemo kasnije da se ta "mašinerija" oslanja na tzv. dinamičku alokaciju memorije).

Nije na odmet napomenuti da za moderne nizove nije u potpunosti podržana semantika pomjeranja (nju zapravo za njih nije ni moguće u potpunosti implementirati, ali razlog za to ne možemo objasniti bez ulaženja u detalje kako interno radi semantika pomjeranja, što ćemo biti u stanju razumjeti tek dosta kasnije). Naime, čak i ukoliko koristimo funkciju "`move`" s modernim nizovima, oni će se kopirati, a ne samo pomjeriti, mada će se njihovi elementi pomjerati umjesto da se kopiraju u slučaju da sami njihovi elementi podržavaju semantiku pomjeranja. Početnici se s ovim detaljima ne trebaju zamarati, ali samo treba znati da funkcija "`move`" s modernim nizovima ni izbliza ne donosi toliko drastične razlike u efikasnosti kao recimo s vektorima.

Na kraju izlaganja o vektorima i modernim nizovima, treba napomenuti da C++ posjeduje još jedan veoma koristan tip podataka nazvan *dek* (engl. *deque*, što je akronim od *double ended queue*). Dekovi se deklariraju uz pomoć riječi "`deque`", na identičan način kao i vektori, pri čemu je za njihovo korištenje potrebno u program uključiti zaglavlje istoimene biblioteke (tj. "`deque`"). Dekovi su funkcionalno veoma srodni vektorima i gotovo sve konstrukcije koje rade s vektorima rade i s dekovima. Često je moguće u čitavom programu riječ "`vector`" prosto zamijeniti s "`deque`" i da sve i dalje radi. U čemu je onda razlika? Najbitnija razlika između vektora i dekova, s aspekta korištenja, je što dekovi pored funkcije "`push_back`", koja omogućava efikasno dodavanje elemenata na *kraj*, podržavaju i funkciju "`push_front`" koja omogućava efikasno dodavanje elemenata na *početak*. Pri tome se ova operacija obavlja *bez premještanja elemenata deka u memoriji*, što znači vrlo efikasno. Pored toga, kod vektora se može desiti da operacije "`push_back`" ili "`resize`" dovedu do pomjeranja elemenata vektora u memoriji, dok kod deka postoji garancija da ni jedna od funkcija "`push_back`", "`push_front`" i "`resize`" neće dovesti do pomjeranja elemenata u memoriji (mada za funkcije "`insert`" i "`erase`" takva garancija ne postoji ni za vektor, ni za dek). U nekim primjenama ovo može biti značajno (npr. ukoliko smo postavili neki pokazivač da pokazuje na neki element vektora, nemamo garanciju da će taj pokazivač biti valjan nakon što izvršimo operaciju "`push_back`", dok takva garancija postoji u slučaju deka). Cijena koja je "plaćena" za ostvarivanje ovih pogodnosti je činjenica da elementi deka nisu nužno smješteni u memoriji jedan za drugim, što *zabranjuje primjenu pokazivačke aritmetike*. Na primjer, ukoliko "`p`" pokazuje na peti element deka, ne postoji nikakva garancija da će "`p + 1`" pokazivati na šesti element deka (moguće je da hoće, ali moguće je i da neće). Iz istog razloga, dekovi ne podržavaju ni funkciju "`data`", s obzirom da se ona uglavnom koristi kada zbog nekog razloga želimo raditi s pokazivačkom aritmetikom. S druge strane, takozvana *iteratorska aritmetika*, o kojoj ćemo ponešto govoriti nešto kasnije, radi ispravno i za vektore i za dekove (iteratori su zapravo jedna vrsta poopćenja pokazivača uvedena u jeziku C++-u, koja omogućava efikasno kretanje kroz kolekcije podataka koji ne moraju nužno biti smješteni u memoriju jedan za drugim). Još jedna nuspojava "razbacanosti" elemenata deka u memoriji je činjenica da je pristup elementima deka osjetno sporiji nego u slučaju nizova odnosno vektora (s obzirom na znatno komplikovanije pronalaženje lokacije gdje se element kojem pristupamo zaista nalazi). Stoga dekove treba koristiti kada nam je bitna mogućnost efikasnog dodavanja novih podataka na oba kraja (tj. na početak odnosno kraj), ili kada nam je bitna nepomičnost elemenata u memoriji prilikom promjene veličine. U svakom slučaju, ostavljena nam je mogućnost izbora da odaberemo onaj tip podataka koji najbolje odgovara problemu koji želimo riješiti.